

JobSheet - Week 5 – Generic Class

Nama : Raza Mulki Firdaus
NIM : 251524120
Kelas : 1D
Repo GitHub : <https://github.com/zaaraza/Teknik-Pemrograman->

Instruksi Pengerjaan:

1. Kerjakan 6 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W4_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W5_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Try First

Soal Praktikum
Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya! <pre>public class GenericsTypeOld { private Object t; public Object get() { return t; } public void set(Object t) { this.t = t; } }</pre>

```

public static void main(String args[]) {
    GenericsTypeOld type = new GenericsTypeOld();
    type.set("Java");
    String str = (String) type.get(); // type casting, error prone and can cause
    ClassCastException
}
}

```

Source Code



```

1  public class GenericsTypeOld {
2      private Object t;
3
4      public Object get() {
5          return t;
6      }
7
8      public void set(Object t) {
9          this.t = t;
10     }
11
12     public static void main(String args[]) {
13         GenericsTypeOld type = new GenericsTypeOld();
14         type.set("Java");
15         String str = (String) type.get(); // casting
16         System.out.println(str);
17     }
18 }

```

Screenshot Hasil

Java

Penjelasan

Program ini menggambarkan cara lama dalam Java sebelum fitur Generics ada, yaitu menggunakan tipe data Object sebagai wadah universal. Karena semua kelas di Java mewarisi dari Object, variabel bertipe Object bisa menyimpan nilai apa pun, baik String, Integer, maupun tipe lainnya.

Masalah utama dari pendekatan ini muncul saat kita ingin mengambil kembali nilai yang sudah disimpan. Karena tipe kembaliannya adalah Object, kita harus melakukan type casting secara manual, misalnya menulis (String) type.get() agar Java tahu bahwa nilai tersebut bertipe String. Ini memang bisa berjalan, tetapi sangat rawan kesalahan. Jika suatu saat kita tidak sengaja menyimpan nilai bertipe Integer lalu mencoba cast ke String, program akan langsung crash saat dijalankan dengan error ClassCastException.

Inilah mengapa pendekatan ini disebut error prone atau rentan kesalahan. Compiler Java tidak bisa membantu mendeteksi kesalahan tipe saat kode ditulis, sehingga bug baru diketahui saat program sudah berjalan.

Nama Teman Hal yang Dibantu (Opsional)

-

No. 2 Using Generic Class

Soal Praktikum

Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

```
public class GenericsType<T> {  
  
    private T t;  
  
    public T get() {  
        return this.t;  
    }  
  
    public void set(T t1) {  
        this.t = t1;  
    }  
  
    public static void main(String args[]) {  
        GenericsType<String> type = new GenericsType<>();  
        type.set("Java"); // valid  
  
        GenericsType type1 = new GenericsType(); // raw type  
        type1.set("Java"); // valid  
        type1.set(10); // valid and autoboxing support  
    }  
}
```

Source Code



```
1  public class GenericsType<T> {  
2      private T t;  
3  
4      public T get() {  
5          return this.t;  
6      }  
7  
8      public void set(T t1) {  
9          this.t = t1;  
10     }  
11  
12     public static void main(String args[]) {  
13         GenericsType<String> type = new GenericsType<>();  
14         type.set("Java"); // valid  
15         GenericsType type1 = new GenericsType(); // raw type (tanpa tipe spesifik)  
16         type1.set("Java"); // valid  
17         type1.set(10); // juga valid, autoboxing int → Integer  
18  
19         // print untuk melihat hasilnya  
20         System.out.println("Nilai type1 setelah set Java: " + type1.get());  
21         System.out.println("Nilai type1 setelah set 10: " + type1.get());  
22         System.out.println("Nilai type (String generic): " + type.get());  
23     }  
24 }
```

Screenshot Hasil

```
Nilai type1 setelah set Java: 10
Nilai type1 setelah set 10: 10
Nilai type (String generic): Java
```

Penjelasan

Program ini adalah versi yang sudah diperbaiki dari soal pertama, yaitu menggunakan fitur Generics yang diperkenalkan sejak Java 5. Dengan menambahkan parameter tipe <T> pada deklarasi kelas, kita bisa menentukan jenis data apa yang akan disimpan sejak awal.

Ketika kita mendeklarasikan `GenericType<String>`, artinya variabel T di dalam kelas tersebut akan diperlakukan sebagai String sepenuhnya. Tidak ada lagi kebutuhan untuk melakukan type casting manual, dan compiler Java akan langsung memberi tahu jika kita mencoba memasukkan tipe yang salah bahkan sebelum program dijalankan.

Di bagian main juga diperlihatkan penggunaan raw type, yaitu `GenericType` tanpa parameter tipe. Ini diizinkan untuk menjaga kompatibilitas dengan kode lama, namun compiler akan memberikan peringatan. Raw type kehilangan manfaat utama generics karena tidak ada pengecekan tipe yang ketat.

Nama Teman Hal yang Dibantu (Opsional)

-

No. 3 Java Generic Class

Soal Praktikum

Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

```
interface MinMax<T extends Comparable<T>> {
    T max();
}

class MyClass<T extends Comparable<T>> implements MinMax<T> {
    T[] vals;

    MyClass(T[] o) {
        vals = o;
    }

    public T max() {
        T v = vals[0];

        for (int i = 1; i < vals.length; i++) {
            // Menggunakan method compareTo dari interface Comparable
            if (vals[i].compareTo(v) > 0) {
                v = vals[i];
            }
        }
        return v;
    }
}

public class Main {
    public static void main(String[] args) {
        Integer[] inums = {3, 6, 2, 8, 6};
        Character[] chs = {'b', 'r', 'p', 'w'};

        MyClass<Integer> a = new MyClass<>(inums);
        MyClass<Character> b = new MyClass<>(chs);

        System.out.println("Nilai Integer Terbesar: " + a.max());
        System.out.println("Karakter Terbesar: " + b.max());
    }
}
```

Source Code

```

1 interface MinMax<T extends Comparable<T>> {
2     T max();
3 }
4
5 class MyClass<T extends Comparable<T>> implements MinMax<T> {
6     T[] vals;
7
8     MyClass(T[] o) {
9         vals = o;
10    }
11
12    public T max() {
13        T v = vals[0];
14        for (int i = 1; i < vals.length; i++) {
15            if (vals[i].compareTo(v) > 0) {
16                v = vals[i];
17            }
18        }
19        return v;
20    }
21 }
22
23 public class Main {
24     public static void main(String[] args) {
25         Integer[] inums = { 3, 6, 2, 8, 6 };
26         Character[] chs = { 'b', 'r', 'p', 'w' };
27
28         MyClass<Integer> a = new MyClass<>(inums);
29         MyClass<Character> b = new MyClass<>(chs);
30
31         System.out.println("Nilai Integer Terbesar: " + a.max());
32         System.out.println("Karakter Terbesar: " + b.max());
33     }
34 }

```

Screenshot Hasil

```

Nilai Integer Terbesar: 8
Karakter Terbesar: w

```

Penjelasan

Program ini memperlihatkan bagaimana Generics bisa dikombinasikan dengan interface dan inheritance untuk membuat kode yang lebih fleksibel namun tetap aman secara tipe. Interface MinMax mendefinisikan kontrak bahwa setiap implementasinya harus bisa mencari nilai maksimum.

Kunci dari program ini adalah batasan `T extends Comparable<T>`. Artinya, tipe `T` yang digunakan harus merupakan tipe yang sudah mengimplementasikan interface `Comparable`, sehingga kita bisa menggunakan method `compareTo` untuk membandingkan dua nilai. `Integer` dan `Character` keduanya sudah memenuhi syarat ini.

Di dalam method `max()`, program menelusuri seluruh elemen array. Setiap elemen dibandingkan dengan nilai terbesar sementara menggunakan `compareTo`. Jika hasilnya lebih dari nol, berarti elemen tersebut lebih besar dan nilai terbesar diperbarui. Berkat Generics, satu kelas yang sama bisa digunakan untuk mencari nilai maksimum dari array `Integer` maupun `Character`.

Nama Teman Hal yang Dibantu (Opsional)

-

No. 4 Java Generic Method

Soal Praktikum

Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

```
public class GenericsMethods {

    // Java Generic Method
    public static <T> boolean isEqual(GenericsType<T> g1, GenericsType<T> g2) {
        return g1.get().equals(g2.get());
    }

    public static void main(String args[]) {
        GenericsType<String> g1 = new GenericsType<>();
        g1.set("Java");

        GenericsType<String> g2 = new GenericsType<>();
        g2.set("Java");

        boolean isEqual = GenericsMethods.<String>isEqual(g1, g2);
        // above statement can be written simply as
        isEqual = GenericsMethods.isEqual(g1, g2);

        /*
         * This feature, known as type inference, allows you to invoke
         * a generic method as an ordinary method, without specifying
         * a type between angle brackets
         */

        // Compiler will infer the type that is needed
    }
}
```

Source Code

```

1 public class GenericsMethods {
2     public static <T> boolean isEqual(GenericsType<T> g1, GenericsType<T> g2) {
3         return g1.get().equals(g2.get());
4     }
5
6     public static void main(String args[]) {
7         GenericsType<String> g1 = new GenericsType<>();
8         g1.set("Java");
9
10        GenericsType<String> g2 = new GenericsType<>();
11        g2.set("Java");
12
13        boolean isEqual = GenericsMethods.<String>isEqual(g1, g2);
14        // bisa juga ditulis lebih simpel:
15        isEqual = GenericsMethods.isEqual(g1, g2);
16
17        System.out.println("Apakah sama? " + isEqual);
18    }
19 }

```

Screenshot Hasil

Apakah sama? true

Penjelasan

Program ini menunjukkan bahwa Generics tidak hanya bisa diterapkan pada kelas, tetapi juga langsung pada sebuah method. Generic method memiliki parameter tipe sendiri yang ditulis sebelum tipe kembaliannya, seperti <T> boolean isEqual(...). Ini memungkinkan method tersebut bekerja dengan berbagai tipe data secara aman tanpa perlu membuat banyak method dengan nama berbeda.

Dalam contoh ini, method isEqual menerima dua objek GenericsType dengan tipe yang sama, lalu membandingkan nilainya menggunakan method equals. Kita bisa memanggil method ini dengan menyebutkan tipe secara eksplisit seperti GenericsMethods.<String>isEqual(g1, g2), atau bisa juga langsung ditulis GenericsMethods.isEqual(g1, g2) tanpa menyebutkan tipe sama sekali.

Kemampuan Java untuk secara otomatis menentukan tipe dari konteks penggunaannya ini disebut type inference. Dengan type inference, kode menjadi lebih ringkas dan mudah dibaca tanpa mengorbankan keamanan tipe. Compiler tetap memverifikasi tipe di balik layar meski kita tidak menuliskannya secara eksplisit di kode.

Nama Teman Hal yang Dibantu (Opsional)

-

No. 5 Java Generics Bounded Type Parameters

Soal Praktikum

Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

```
class Bound<T extends A> {
```

```

private T objRef;

public Bound(T obj) {
    this.objRef = obj;
}

public void doRunTest() {
    this.objRef.displayClass();
}
}

class A {

    public void displayClass() {
        System.out.println("Inside super class A");
    }
}

class B extends A {

    public void displayClass() {
        System.out.println("Inside sub class B");
    }
}

class C extends A {

    public void displayClass() {
        System.out.println("Inside sub class C");
    }
}

public class BoundedClass {

    public static void main(String a[]) {

        // Creating object of sub class C and
        // passing it to Bound as a type parameter.
        Bound<C> bec = new Bound<C>(new C());
        bec.doRunTest();

        // Creating object of sub class B and
        // passing it to Bound as a type parameter.
        Bound<B> beB = new Bound<B>(new B());
        beB.doRunTest();

        // similarly passing super class A
    }
}

```

```

        Bound<A> bea = new Bound<A>(new A());
        bea.doRunTest();
    }
}

```

Source Code

```

1  class A {
2      public void displayClass() {
3          System.out.println("Inside super class A");
4      }
5  }
6
7  class B extends A {
8      public void displayClass() {
9          System.out.println("Inside sub class B");
10     }
11 }
12
13 class C extends A {
14     public void displayClass() {
15         System.out.println("Inside sub class C");
16     }
17 }
18
19 class Bound<T extends A> {
20     private T objRef;
21
22     public Bound(T obj) {
23         this.objRef = obj;
24     }
25
26     public void doRunTest() {
27         this.objRef.displayClass();
28     }
29 }
30
31 public class BoundedClass {
32     public static void main(String a[]) {
33         Bound<C> bec = new Bound<C>(new C());
34         bec.doRunTest();
35
36         Bound<B> beB = new Bound<B>(new B());
37         beB.doRunTest();
38
39         Bound<A> bea = new Bound<A>(new A());
40         bea.doRunTest();
41     }
42 }

```

Screenshot Hasil

```

Inside sub class C
Inside sub class B
Inside super class A

```

Penjelasan

Program ini memperkenalkan konsep bounded type parameter, yaitu cara membatasi tipe generik agar hanya menerima kelas tertentu beserta turunannya. Kelas Bound<T extends A> berarti T hanya boleh berupa kelas A itu sendiri, atau kelas

lain yang mewarisi dari A, seperti B dan C. Jika kita mencoba menggunakan tipe lain seperti String atau Integer, compiler akan langsung menolaknya.

Manfaat utama dari pembatasan ini adalah kita bisa dengan aman memanggil method milik kelas A melalui objek bertipe T. Dalam method `doRunTest()`, program memanggil `this.objRef.displayClass()`. Karena compiler sudah mengetahui bahwa T pasti merupakan A atau subkelasnya, panggilan ini dijamin valid dan tidak akan menyebabkan error saat kompilasi maupun saat program berjalan.

Saat program dijalankan, yang terjadi adalah polymorphism. Meski method yang dipanggil bernama `displayClass`, output yang muncul berbeda-beda tergantung objek aslinya. Jika objeknya adalah B maka mencetak pesan B, jika C maka mencetak pesan C. Hal ini karena method `displayClass` sudah di-override di masing-masing subkelas, sehingga Java secara otomatis memanggil versi yang sesuai dengan tipe objek yang sebenarnya.

Nama Teman Hal yang Dibantu (Opsional)

-

No. 6 Java Generic WildCard

Soal Praktikum

Silahkan mencoba source code pada lampiran dan memberikan penjelasan detail setiap bagian pentingnya!

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.HashSet;
import java.util.LinkedList;

/**
 * Wildcard Arguments With An Unknown Type
 * @author javaguides.net
 */
public class WildCardSimpleExample {

    public static void printCollection(Collection<?> c) {
        for (Object e : c) {
            System.out.println(e);
        }
    }

    public static void main(String[] args) {
        Collection<String> collection = new ArrayList<>();
        collection.add("ArrayList Collection");
        printCollection(collection);
    }
}
```

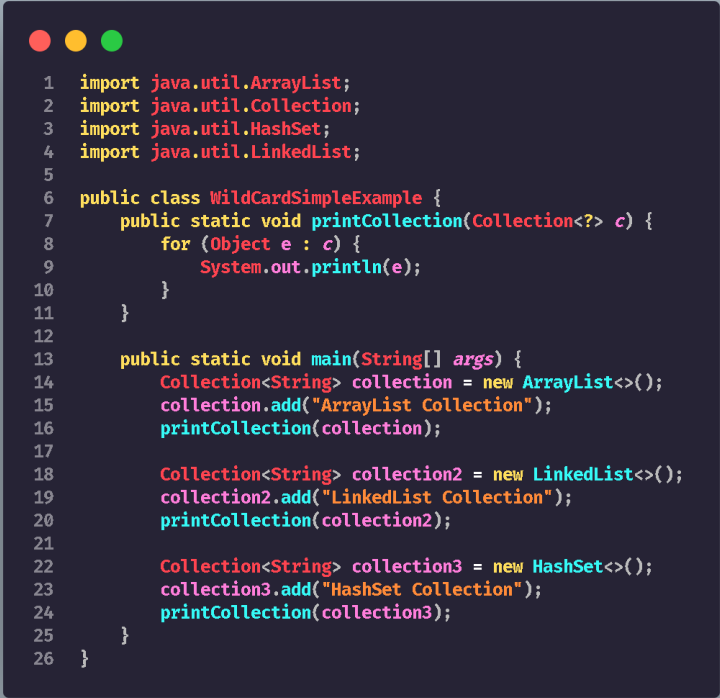
```

Collection<String> collection2 = new LinkedList<>();
collection2.add("LinkedList Collection");
printCollection(collection2);

Collection<String> collection3 = new HashSet<>();
collection3.add("HashSet Collection");
printCollection(collection3);
}
}

```

Source Code



```

1 import java.util.ArrayList;
2 import java.util.Collection;
3 import java.util.HashSet;
4 import java.util.LinkedList;
5
6 public class WildCardSimpleExample {
7     public static void printCollection(Collection<?> c) {
8         for (Object e : c) {
9             System.out.println(e);
10        }
11    }
12
13    public static void main(String[] args) {
14        Collection<String> collection = new ArrayList<>();
15        collection.add("ArrayList Collection");
16        printCollection(collection);
17
18        Collection<String> collection2 = new LinkedList<>();
19        collection2.add("LinkedList Collection");
20        printCollection(collection2);
21
22        Collection<String> collection3 = new HashSet<>();
23        collection3.add("HashSet Collection");
24        printCollection(collection3);
25    }
26 }

```

Screenshot Hasil

```

ArrayList Collection
LinkedList Collection
HashSet Collection

```

Penjelasan

Program ini memperkenalkan konsep wildcard dalam Generics Java, yang direpresentasikan dengan tanda tanya (?). Wildcard digunakan ketika kita ingin membuat sebuah method yang bisa menerima koleksi dengan tipe elemen apa pun, tanpa perlu menentukan tipe spesifiknya terlebih dahulu.

Alasan mengapa kita tidak cukup menulis `Collection<Object>` adalah karena dalam aturan Generics Java, `Collection<String>` bukanlah subtype dari `Collection<Object>` meskipun `String` itu sendiri adalah subtype dari `Object`. Inilah yang membuat wildcard menjadi sangat penting. Dengan menulis `Collection<?>`, method `printCollection` bisa menerima `ArrayList`, `LinkedList`, maupun `HashSet` dengan tipe elemen apa pun tanpa ada batasan.

Di dalam method, elemen-elemen koleksi diperlakukan sebagai Object karena tipe pastinya tidak diketahui saat kompilasi. Wildcard ini bersifat read-only, artinya kita tidak bisa menambahkan elemen baru ke dalam koleksi melalui method ini. Program ini sekaligus menunjukkan polimorfisme dari interface Collection, di mana satu interface yang sama diimplementasikan oleh beberapa kelas berbeda namun bisa diperlakukan secara seragam.

Nama Teman Hal yang Dibantu (Opsional)
-